

Lab Assignment: OSINT & Blockchain Forensics (Solana Investigation)

Case Study: TOKTHEDOG (\$TOK)

Learning Objectives

- **Understand the Solana blockchain architecture** (Mint Address, Accounts, Liquidity Pools, SPL Tokens).
- **Pivot from on-chain identifiers** to Web2 digital footprints (Telegram, Twitter/X, websites).
- **Trace financial flows** and map associated wallets (Deployer, Liquidity Provider, Top Holders).
- **Automate metadata collection** and address tracking using Python and REST APIs.

Scenario & Input Data

A memecoin named **TOKTHEDOG (\$TOK)** is under forensic investigation. You only have the following partial on-chain identifiers:

Entity	Solana Address
Token Mint Address (\$TOK)	7fuinNNcyXXXXXXXXXXXXXXXX1BVCnVwoXeVQ4cXP
Pair Address (Liquidity Pool)	E5ihhxqKYnzpXXXXXXXXXXwpQAHmtTKJhKZMWN8v9
Wrapped SOL Mint	So111111111111XXXXXXXXXXXX111112

Part I — Theoretical Review: Solana Mechanisms (1h 30m)

1. SPL Tokens vs. ERC-20 Structure

- **Standard:** Solana uses the SPL (Solana Program Library) standard.
- **Mint Address:** Represents the master contract/identifier of the token (equivalent to an ERC-20 smart contract address).
- **Token Account (ATA):** Unlike Ethereum where an address directly holds a token balance, Solana requires an **Associated Token Account (ATA)** specific to each token held by a wallet.
- **Liquidity Pool (Pair):** An account owned by a DEX (such as Raydium, Pump.fun, or Orca) holding reserves of two tokens (e.g., \$TOK and SOL) to facilitate swaps.

2. Theoretical Validation Questions

1. What is the technical difference between a **Mint Address** and a **Pair Address**?

2. Why is the **liquidity creation transaction** critical for identifying the original developer (Deployer)?

Part II — OSINT & Digital Footprint Collection (2h 00m)

1. Web2 Pivots via DEX Aggregators

Use aggregators such as **DexScreener**, **GeckoTerminal**, or **Birdeye** by searching with the Pair or Mint address.

Tasks:

- Extract the official token description stored in Metaplex metadata.
- Identify all social media handles linked to the token.

Data Field	Target Identification
Description / Lore	Extract official metadata summary.
Official Telegram	Identify linked channels/groups.
Twitter / X Mentions	Linked community relay accounts.
Launch Platform	Detect if the token originated from a bonding-curve launchpad (e.g., Pump.fun) or direct DEX pool creation.

2. Social Media Forensics

- **Telegram:** Use tools like Telemetrio or TGStat to inspect channel history (creation date, subscriber growth, edited messages).
- **Twitter / X:** Search exact contract addresses and tickers to pinpoint the initial user account that posted the contract address (CA).

Part III — On-Chain Traceability & Forensics (2h 00m)

Use a block explorer such as **Solscan.io** or **SolanaFM**.

1. Identifying the Deployer

1. Open the Mint address page on Solscan.
2. Navigate to **Transactions** / **Instruction History** and locate the earliest transaction (Genesis / Mint To).
3. Identify the signer wallet address (**Signer**), which corresponds to the **Deployer**.

2. Funding Source Analysis (Inflow)

Inspect the Deployer's earliest SOL transactions to identify how the initial wallet was funded:

- **Case A (Centralized Exchange):** Funds received directly from CEX hot wallets (Binance, Bybit, Coinbase) → Potential KYC subpoena target.
- **Case B (Mixer / Cross-Chain Bridge):** Funds routed via FixedFloat, ChangeNOW, Maya Protocol, etc.
- **Case C (Secondary Wallet):** Funds transferred from an intermediary personal wallet.

3. Network & Liquidity Analysis

1. Inspect the Liquidity Pool address `E5ihhxqKYnzpXXXXXXXXXXwpQAHmtTKJhKZMWN8v9`.
2. Verify liquidity status: Is liquidity **Burned** or **Locked**?
3. Perform an analysis of the **Top 10 Holders**: Check for abnormal wallet clustering (e.g., multiple top wallets created within the same minute receiving supply via bundled transactions).

Part IV — Python Automation & API Integration (1h 30m)

Base Metadata Extraction Script

Python

```
import requests
import json

def analyze_solana_token(token_mint):
    url = f"https://api.dexscreener.com/latest/dex/tokens/{token_mint}"
    headers = {"User-Agent": "Mozilla/5.0"}

    response = requests.get(url, headers=headers)
    if response.status_code != 200:
        print(f"API Error: {response.status_code}")
        return

    data = response.json()
    pairs = data.get("pairs", [])
    if not pairs:
        print("No pairs found.")
        return

    print(f"=== OSINT REPORT: {token_mint} ===")
```

```

main_pair = pairs[0]
print(f"Name: {main_pair.get('baseToken', {}).get('name')}")
print(f"Symbol: {main_pair.get('baseToken', {}).get('symbol')}")
print(f"USD Price: {main_pair.get('priceUsd')}")
print(f"DEX: {main_pair.get('dexId')}")
print(f"Pair Address: {main_pair.get('pairAddress')}")

info = main_pair.get("info", {})
websites = info.get("websites", [])
socials = info.get("socials", [])

print("\n--- Web Links & Social Networks ---")
for site in websites:
    print(f"Website ({site.get('label', 'Site')}): {site.get('url')}")
for social in socials:
    print(f"Social ({social.get('type')}): {social.get('url')}")

if __name__ == "__main__":
    TOKEN_MINT = "7fuinNNcyLmJh2eZKhVB6p5FTcBjiBVCnVwoXeVQ4cXP"
    analyze_solana_token(TOKEN_MINT)
```[cite: 1]

Practical Exercise
1. Run the script to inspect the raw JSON response[cite: 1].
2. Modify the script to query the public RugCheck API (https://api.rugcheck.xyz/v1/tokens/ {mint}/report`) to automatically extract the Risk Score and check the status of Mint Authority and Freeze Authority[cite: 1].

Submission Deliverables

Submit a single structured investigation report containing[cite: 1]:
1. Digital Identity Sheet: Verified social links, lore, and active trading platforms[cite: 1].
2. On-Chain Genealogy Tree: Deployer address $\rightarrow$ Inflow funding source $\rightarrow$ Pool creation address[cite: 1].
3. Project Legitimacy Assessment: Liquidity lock verification and holder cluster analysis[cite: 1].
4. Automated Python Script: Working .py file generating the automated report[cite: 1].

```

